Northwind 2.0 Developer Edition: Purchase Orders

Ala Arman

▶ *Applies To*

Select any topic below to learn about Purchase Orders in the Northwind Developer Edition.

Overview

This Dev Edition of the Northwind Purchase Orders sample application has the only Purchase Orders module. In Starter Edition, products never run out and never need to be purchased. This Dev Edition expands on the database schema (tables used) in Northwind 2.0 Starter Edition and uses more advanced features. It is still meant to introduce you to the major features of Microsoft Access, not to run any specific business.

Using Purchase Orders

- The Purchase Order List is available from the Ribbon. It has hyperlinks to open each purchase order.
- Both the Purchase Order List and the Ribbon have a New Purchase Order button to open a new, blank purchase order. You can also create Purchase Orders from the **Products form >Reorder Product** button.
- The buttons in the header advance the Purchase Orders through the workflow through **Submit, Approve**, **Receive** and **Close**. The corresponding tracking fields on the form are locked because they should only be set programmatically by clicking the action buttons.
- Approving a PO requires Purchase Approval privilege. You can login as Andrew Cencini, who has the privilege, or give it to yourself in **System Admin > Privileges**. Note that your ability to do so is one of the many reasons Northwind is not a production quality application. In the real world, users will not be able to

Was this information helpful? Yes No

- Line items in a Purchase Order are validated with respect to their Quantity. It must be at least the Minimum ReOrder Quantity, and ideally it should get inventory back up to at least the Target Level, as set for each Product.

- When a Purchase Order is **Received**, special processing is invoked to distribute those products to order line items that are in **No Stock** status and set them to **Allocated**. Any remaining quantity is sent to inventory. A record is added to the **StockTake** table.

Programming Purchase Orders

This section addresses noteworthy implementation details of the Purchase Order form, **frmPurchaseOrderDetails**:

1. The Purchase Order form gets its data from a simple query, **qryPurchaseOrder** (see **RecordSource** property). Basing a data entry form on a simple query is a best practice. Note that it is not necessary to include the **PurchaseOrderDetails** table in this query. Details are handled by the subform. But the query does join with other tables to pick up the read-only **StatusName**, **SubmittedBy**, and **ApprovedBy** fields.

2. The **PurchaseOrderList** form can open multiple instances of the Purchase Order form. This is handy because the PO department deals with lots of interruptions and may need to open another PO while working on the first one -- or compare it to a third PO. The technique is documented here.

3. **VendorID** gets its value from a two-column combobox: a hidden ID column, and a visible Description column. Such comboboxes are bound to simple two-column queries: see the **RowSource** property.

4. When saving a record, at the very least the *required* fields must be filled out. In Starter edition, we let the Access default behavior happen; in this Dev edition, a more user-friendly technique is implemented -- as described in detail below.

5. When the **PO Status** goes to **Received**, special processing is invoked (procedure **AllocateToInventory**) to distribute the new inventory over orders that are waiting for these products.

VALIDATION

The validation code implemented in Northwind Dev edition requires only three lines of code:

Was this information helpful?

- In **Form_Current**: *ValidateForm_RemoveHighlights Me*

This is a good pattern to follow: making your code very self-contained makes it easy to implement everywhere. Professional developers may take this even further, for example using form subclassing. (This is beyond the goals for Northwind Dev.)

The self-contained validation code accepts a form object to validate. It then checks the underlying **RecordsetClone**'s form collection to find out which controls are bound to required fields, and checks if they have a value. If they don't, they are highlighted.

Learn more

- Northwind 2.0 Developer Edition: Things you should know.
- Northwind 2.0 Developer Edition: All topics

Need more help?

How can we help you?

Want more options?

Discover **Community**

Explore subscription benefits, browse training courses, learn how to secure your device, and more.

Was this information helpful?



Microsoft 365 subscription benefits



Microsoft 365 training



Microsoft security



Accessibility center

What's new

Surface Pro

Surface Laptop

Surface Laptop Studio 2

Copilot for organizations

Copilot for personal use

AI in Windows

Explore Microsoft products

Windows 11 apps

Microsoft Store

Account profile

Download Center

Microsoft Store support

Returns

Order tracking

Certified Refurbished

Microsoft Store Promise

Flexible Payments

Education

Microsoft in education

Devices for education

Microsoft Teams for Education

Microsoft 365 Education

How to buy for your school

Educator training and development

Deals for students and parents

AI for education

Business

Microsoft AI

Microsoft Security

Dynamics 365

Microsoft 365

Microsoft Power Platform

Microsoft Teams

Microsoft 365 Copilot

Small Business

Developer & IT

Azure

Microsoft Developer

Microsoft Learn

Support for AI marketplace apps

Microsoft Tech Community

Microsoft Marketplace

Marketplace Rewards

Visual Studio

Company

Careers

About Microsoft

Company news

Privacy at Microsoft

Investors

Diversity and inclusion

Accessibility

Sustainability

English (United States) Your Privacy Choices Consumer Health Privacy

Sitemap Contact Microsoft Privacy Terms of use Trademarks Safety & eco Recycling About our ads

© Microsoft 2026

Was this information helpful?